

Formato base. JSON

El formato JSON se eligió principalmente debido a su versatilidad de manejo de archivos y su simple transcripción a nuestro sistema. Tal vez no se pueda observar de manera fácil a la vista o de ejecución la eficiencia de este tipo de archivos respecto a por ejemplo al CSV y su ordenamiento de datos debido a que no manejamos una gran cantidad de datos, pero si los manejamos de manera rápida. Explicado esto podemos definir esto mediante los siguientes criterios:

- **Tamaño y legibilidad**

Su uso de datos de “Clave/valor” nos facilita la búsqueda de datos específicos así como su rápida lectura de archivos, y viendo una lectura humana podemos visualizarlo sencillamente con un editor de texto.

- **Parseo**

Python tiene una característica especial y manejo sencillo con la librería nativa JSON con funciones sencillas como el `json.dump`

- **Robustez**

A diferencia de un txt o CSV donde no podemos leer un archivo sin de antemano saber nosotros y convertir cada tipo de dato debido a que son string, el archivo JSON nos guarda de base el tipo de archivo que manejamos.

Escritura y Respaldo

Para el correcto traspaso de guardado de archivo y evitar cualquier acción accidental usamos un tipo de archivo temporal (.temp) en donde si tenemos una configuración anterior la guardamos como un .bak de uso de respaldo o copia de seguridad, así haciendo que el archivo temporal se convierta en el nuevo archivo base `config.json`.

Todo esto esta principalmente manejado con la función llamada “GUARDAR_CONFIG” en la carpeta de LogicaYDatos en el archivo `config.py`:

```

29 def GUARDAR_CONF(nuevos_datos):#Guardamos configuración con una sobrescritura con open("w") y utf-8
30     try:
31         with open(TEMPORAL, "w", encoding="utf-8") as f:
32             json.dump(nuevos_datos, f, indent=4, ensure_ascii=False)
33
34         if os.path.exists(ARCHIVO): #Si nuestro archivo base ya existe
35             os.replace(ARCHIVO, RESPALDO) #Lo reemplazamos con nuestro respaldo
36
37             os.replace(TEMPORAL, ARCHIVO)#Y así nuestro archivo temporal será reemplazado con nuestro archivo base
38         return True
39     except PermissionError:
40         return False

```

Manejo de errores.

- Archivo ausente

Con un sencillo “FileNotFoundError” se agarrará este error que nos da a entender que no tenemos ningún archivo para su lectura de configuración y aplicación visual. Tenemos unos datos llamados “VALORES_DEFECTO” con datos base para evitar este tipo de problemas.

- Archivo corrupto

Usando la función nativa `json.JSONDecodeError` en donde si el archivo json se modificó en sintaxis válida al momento de hacer el `json.load()` nos obligará nuevamente a usar los valores por defecto.

- Sin permisos

Con `PermissionError` buscaremos que si el sistema bloquea la lectura se aplican valores por defecto y en caso de guardar los datos y no tener el permiso por alguna razón de guardado simplemente mostrar un error en una ventana.

Como se ve, principalmente todas las opciones retornarán los valores por defecto en la lectura de configuración:

```

def LEER_CONF(): #Con un open("r") y encoding utf-8 cargaremos en lectura nuestro archivo base
    try:
        with open(ARCHIVO, "r", encoding="utf-8") as f:
            return json.load(f)
    except (FileNotFoundError, json.JSONDecodeError, PermissionError): #Excepciones a no encontrar el archivo
        #El archivo no sea JSON o error de permiso
        return VALORES_DEFECTO

```

Archivo JSON

```
{ } config.json > ...  
1 {  
2   "usuario": "Andres Aguilón",  
3   "tema": "oscuro",  
4   "idioma": "en-US",  
5   "tam_fuente": 14,  
6   "color_menu": "#00ff00",  
7   "color_letra": "#ff8000",  
8   "foto_perfil": "C:/Users/ADMIN/Desktop/Miguelito Buki.jpg"  
9 }
```

Funcionamiento de la aplicación

